

DNS / DNSS con Filtro de Kalman Adaptativo

τ_t via Rolling Grid Search · MLE Multi-Start · RTS Smoother
Análisis Post-Ajuste Completo



Pipeline de estimacion

Nelson-Siegel (DNS)

$k = 3$ factores

$\tau_{1,t}$ adaptativo

Grid 1D

Nelson-Siegel-Svensson (DNSS)

$k = 4$ factores

$(\tau_{1,t}, \tau_{2,t})$ adaptativos

Grid 2D

Nelson & Siegel (1987) · Diebold & Li (2006) · Svensson (1994)
Caldeira, Laurini & Portugal (2010) · Cakmaklı (2013) · Hamilton (1994)

Contents

1. Introduccion y motivacion

1.1. Que resuelve este notebook

El modelado de curvas de rendimiento con Nelson-Siegel (NS) y Nelson-Siegel-Svensson (NSS) plantea dos problemas de estimacion:

1. **El parametro de forma τ es no lineal.** Las cargas $H(\tau)$ dependen de τ de forma no lineal, lo que impide estimar los factores por OLS con τ fijo sin perder informacion. La solucion clasica de Diebold & Li (2006) fija τ en un valor unico para toda la muestra, asumiendo que la forma de la curva no cambia en el tiempo.
2. **Los factores son latentes y correlacionados en el tiempo.** El modelo dinamico de Diebold & Li (2006) trata los factores como un VAR(1), que puede estimarse eficientemente con el Filtro de Kalman. Esto produce estimados de estado optimos (minima varianza, insesgados) bajo los supuestos del modelo.

Este notebook implementa una solucion que ataca ambos problemas simultáneamente:

- τ_t se re-estima **en cada periodo** via Rolling Grid Search, haciendolo adaptativo en el tiempo.
- Los factores β_t se estiman con el **Filtro de Kalman** y el **Suavizador RTS**, con los parametros del proceso de estado estimados por **MLE multi-start**.

1.2. Diferencia fundamental con el enfoque bayesiano

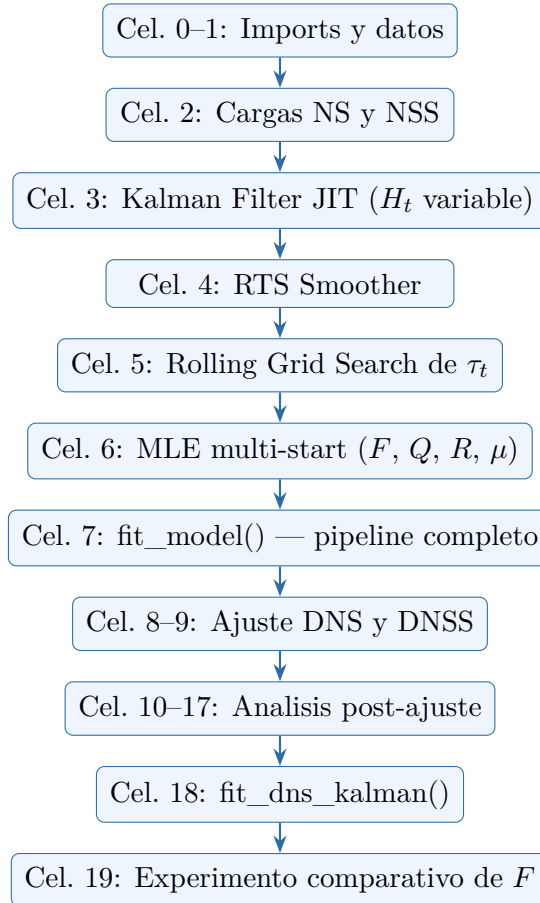
Table 1: Comparacion KF adaptativo vs Gibbs Sampler bayesiano

Aspecto	KF + MLE (este notebook)	Gibbs Sampler (notebook Bayes)
τ en el tiempo	τ_t cambia cada periodo via Grid Search	τ global fijo (estatico) o AR(1) latente
Factores β_t	Estimado puntual $\mathbb{E}[\beta_t \mathbf{Y}_{1:T}]$ via KF+RTS	Distribucion completa $p(\beta_{1:T} \mathbf{Y})$ via FFBS
Parametros θ	MLE numerico (L-BFGS-B)	Posteriors conjugadas (Gibbs)
Incertidumbre	Solo la del estado ($P_{t T}$ del RTS)	Parametrica + de estado + de τ
Velocidad	Muy rapido: $O(T \cdot N^2 \cdot k^2)$	Lento: $O(n_{\text{iter}} \cdot T \cdot N^2 \cdot k^2)$
Intervalos	Basados en la Gaussiana del RTS	Intervalos de credibilidad exactos

1.3. Pipeline de estimacion

- Algoritmo 1.1** (Pipeline DNS/DNSS con KF adaptativo). 1. **Rolling Grid Search** sobre τ : seleccionar τ_t que minimiza el SSE de OLS en cada periodo, usando una ventana de W observaciones.
2. **Construir H_t** : formar el array $(T \times N \times k)$ de cargas variables usando la secuencia $\{\tau_t\}_{t=1}^T$ estimada en el paso 1.
3. **MLE multi-start** (L-BFGS-B): maximizar $\log \mathcal{L}(\boldsymbol{\theta} \mid \mathbf{Y}, H_t)$ sobre $\boldsymbol{\theta} = (\boldsymbol{\mu}, \mathbf{F}, \mathbf{Q}, \mathbf{R})$.
4. **Kalman Filter** (forward): computar $\hat{\boldsymbol{\beta}}_{t|t}$, $\mathbf{P}_{t|t}$ y la log-verosimilitud total.
5. **RTS Smoother** (backward): computar $\hat{\boldsymbol{\beta}}_{t|T}$ y $\mathbf{P}_{t|T}$ usando toda la muestra.
6. **Analisis post-ajuste**: factores, τ_t , curvas ajustadas, residuos, diagnosticos.

1.4. Estructura del notebook



2. Modelo espacio-estado

2.1. Formulacion general

Modelo espacio-estado DNS/DNSS con H_t variable

Ecuacion de observacion:

$$\mathbf{y}_t = \mathbf{H}(\tau_t) \boldsymbol{\beta}_t + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad t = 1, \dots, T \quad (1)$$

Ecuacion de transicion:

$$\boldsymbol{\beta}_t = \boldsymbol{\mu} + \mathbf{F}(\boldsymbol{\beta}_{t-1} - \boldsymbol{\mu}) + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \quad (2)$$

donde $\mathbf{y}_t \in \mathbb{R}^N$ son rendimientos en N plazos, $\boldsymbol{\beta}_t \in \mathbb{R}^k$ son factores latentes ($k = 3$ DNS, $k = 4$ DNSS), $\mathbf{H}_t = \mathbf{H}(\tau_t) \in \mathbb{R}^{N \times k}$ son cargas **variables en el tiempo** (la clave del metodo), $\boldsymbol{\mu} \in \mathbb{R}^k$ es la media de largo plazo de los factores, $\mathbf{F} \in \mathbb{R}^{k \times k}$ la matriz de transicion, $\mathbf{R} = \text{diag}(r_1, \dots, r_N)$ y $\mathbf{Q} = \text{diag}(q_1, \dots, q_k)$ matrices diagonales.

Innovacion central: H_t variable

A diferencia del Filtro de Kalman estandar donde $\mathbf{H}$ es constante, aqui $\mathbf{H}_t = \mathbf{H}(\tau_t)$ cambia en cada periodo porque τ_t fue seleccionado por Grid Search en ese periodo. Esto requiere que el KF reciba un array $(T \times N \times k)$ en lugar de una matriz fija $(N \times k)$. El algoritmo sigue siendo lineal y gaussiano en cada t dado τ_t , por lo que las ecuaciones del KF son exactas.

2.2. Cargas Nelson-Siegel (DNS, $k = 3$)

La funcion de carga de NS mapea cada plazo m_n a tres pesos:

$$\mathbf{H}_n(\tau_1) = \left[1, \quad \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1}, \quad \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1} - e^{-m_n/\tau_1} \right] \quad (3)$$

- $\beta_{0,t}$: **nivel** — carga constante = 1; determina la tasa de largo plazo $\lim_{m \rightarrow \infty} y_t(m) = \beta_{0,t}$.
- $\beta_{1,t}$: **pendiente** — carga decreciente de 1 a 0; captura el spread largo-corto $y_t(\infty) - y_t(0^+) = \beta_{0,t} - (\beta_{0,t} + \beta_{1,t})$.
- $\beta_{2,t}$: **curvatura** — carga en forma de campana con maximo en $m^* \approx 1.793 \tau_1$.

2.3. Cargas Nelson-Siegel-Svensson (DNSS, $k = 4$)

Svensson (1994) extiende NS con un cuarto factor de curvatura independiente:

$$\mathbf{H}_n(\tau_1, \tau_2) = \left[\mathbf{H}_n(\tau_1), \quad \frac{1 - e^{-m_n/\tau_2}}{m_n/\tau_2} - e^{-m_n/\tau_2} \right] \quad (4)$$

$\beta_{3,t}$ es una segunda curvatura con hump en $m^{**} \approx 1.793 \tau_2$. Esto permite capturar formas de curva con dos jorobas o primas de plazo en el tramo largo que NS no puede representar.

Restriccion de separacion: se impone $|\tau_1 - \tau_2| \geq 0.15$ para evitar colinealidad en las columnas de $\mathbf{H}$ que causaria mal condicionamiento en el OLS del Grid Search.

2.4. Interpretacion de τ y el hump

El parametro τ controla donde se ubica el maximo de la carga de curvatura:

$$m^*(\tau) \approx 1.793 \tau \quad \text{anos} \quad (5)$$

Para curvas de Tesoros de EEUU, el hump tipicamente se ubica entre 2 y 5 anos ($\tau_1 \approx 1.1-2.8$). Para TES Colombia el rango es similar aunque puede variar con los ciclos de politica monetaria del Banco de la Republica. El Grid Search re-estima τ_t en cada periodo para seguir estos cambios.

2.5. Estructura de la matriz $\mathbf{F}$

Table 2: Estructuras disponibles para la matriz de transicion $\mathbf{F}$

F_type	Parametros	Interpretacion economica	Referencia
'diagonal'	k	Cada factor sigue un AR(1) independiente. Ortogonalidad de los shocks.	Caldeira et al. (2010) P7, Cakmaklı (2013)
'triangular'	$k(k+1)/2$	Nivel influye en pendiente y curvatura (jerarquia causal). Off-diagonals libres en el triangulo inferior.	Extension propia
'completa'	k^2	VAR(1) irrestricto. Maxima flexibilidad. Requiere verificar estabilidad ($ \lambda_i < 1$).	VAR(1) general

Eleccion de F_type

La literatura empirica (Caldeira et al. 2010, Cakmaklı 2013) reporta diferencias de RMSE < 0.4 pb entre la especificacion diagonal (P7) y el VAR completo (P6). La diagonal es preferible por parsimonia y menor riesgo de sobreajuste, especialmente en muestras de tamano moderado. Para TES Colombia con $T \approx 150-300$ observaciones mensuales, se recomienda empezar con 'diagonal'.

3. Rolling Grid Search de τ_t

3.1. Fundamento: por que re-estimar τ en el tiempo

El enfoque de Diebold & Li (2006) fija τ en un valor unico (tipicamente $\tau = 1.5$ anos) para toda la muestra. Esto es incorrecto cuando la forma de la curva cambia sistematicamente, como ocurre en periodos de cambio de regimen monetario.

El Grid Search por ventana estima, para cada periodo t :

$$\hat{\tau}_t = \arg \min_{\tau \in \mathcal{G}} \text{SSE}(\tau; \mathcal{W}_t) \quad (6)$$

donde $\mathcal{G}$ es la cuadrícula de candidatos y $\mathcal{W}_t$ es la ventana de observaciones usada para evaluar el ajuste.

3.2. Reformulacion via Residual Maker

El SSE de minimos cuadrados de NS en la ventana $\mathcal{W}_t$ puede reescribirse de forma que se *desacopla* el calculo entre τ (dependiente del candidato) y los datos (dependientes del periodo):

Reformulacion del SSE con Residual Maker

Para un candidato τ fijo, el SSE de OLS sobre la ventana $\mathcal{W}_t$ es:

$$\text{SSE}(\tau, t) = \text{tr}(M_\tau \Sigma_t) \quad (7)$$

donde:

$$M_\tau = \mathbf{I} - \mathbf{H}(\tau) [\mathbf{H}(\tau)^\top \mathbf{H}(\tau)]^{-1} \mathbf{H}(\tau)^\top \quad (\text{residual maker, } (N \times N)) \quad (8)$$

$$\Sigma_t = \sum_{s \in \mathcal{W}_t} \mathbf{y}_s \mathbf{y}_s^\top \quad (\text{matriz acumulada de productos cruzados, } (N \times N)) \quad (9)$$

M_τ es el proyector ortogonal al espacio columna de $\mathbf{H}(\tau)$. Es simetrico ($M_\tau = M_\tau^\top$) e idempotente ($M_\tau^2 = M_\tau$).

Eficiencia computacional:

- M_τ se precalcula **una sola vez** por candidato $\tau \in \mathcal{G}$, antes del bucle temporal.
- Σ_t se actualiza incrementalmente mediante **sumas de prefijo acumuladas**: $\Sigma_t = \Sigma_{t-1} + \mathbf{y}_t \mathbf{y}_t^\top$ (ventana expandiente) o $\Sigma_t = \Sigma_{t-1} + \mathbf{y}_t \mathbf{y}_t^\top - \mathbf{y}_{t-W} \mathbf{y}_{t-W}^\top$ (ventana rodante de tamaño W).
- La evaluacion del SSE por candidato en cada periodo es simplemente $\text{tr}(M_\tau \Sigma_t)$, implementada con `einsum` en NumPy:

Celda 5 --- vectorizacion con einsum

```
1 # M_arr : (G, N, N) residual makers precomputados
2 # Sigma : (N, N) matriz de productos cruzados de la ventana actual
3
4 # Evalua SSE para todos los candidatos simultaneamente:
5 sse_vec = np.einsum('gij,ji->g', M_arr, Sigma) # (G,) en O(G*N^2)
6 best_g = np.argmin(sse_vec)
7 tau1_t = tau1_grid[best_g]
```

Complejidad computacional: La formulacion con residual maker reduce la complejidad de $O(|\mathcal{G}| \cdot T \cdot (Nk^2 + N^2k))$ (OLS en cada t) a $O(|\mathcal{G}| \cdot N^2)$ de precalculo + $O(|\mathcal{G}| \cdot T \cdot N^2)$ de evaluacion, con la actualización de Σ_t en $O(N^2)$ por periodo.

3.3. Ventana rodante vs expandiente

La ventana $\mathcal{W}_t$ determina que datos se usan para elegir τ_t :

Table 3: Tipos de ventana para el Grid Search

window_size	Datos usados	Ventajas	Limitaciones
None	$[1, t]$ — toda la historia	Causalmente correcta; τ_t estable; optima para pronostico	Poca reactividad a cambios recientes
W entero	$[t - W + 1, t]$ — ultimas W obs	Reactiva a cambios de regimen; captura estructuras locales	Mayor varianza de τ_t ; sensible a outliers

Recomendacion practica: Para datos semanales de TES Colombia, usar $W \approx 52$ (un ano) como punto de partida. Para datos mensuales, $W \approx 36-60$. Explorar la sensibilidad a W mediante el perfil SSE (disponible en `plot_perfil_sse_tau()`).

3.4. Grid Search 2D para DNSS

Para DNSS, la cuadrícula es bidimensional $\mathcal{G}_1 \times \mathcal{G}_2$ y la reformulacion del SSE requiere el residual maker conjunto:

$$M_{\tau_1, \tau_2} = \mathbf{I} - \mathbf{H}(\tau_1, \tau_2) [\mathbf{H}(\tau_1, \tau_2)^\top \mathbf{H}(\tau_1, \tau_2)]^{-1} \mathbf{H}(\tau_1, \tau_2)^\top \quad (10)$$

El array de residual makers tiene dimension $(G_1 \times G_2 \times N \times N)$ y la evaluacion vectorizada usa:

Celda 5 --- Grid Search 2D para DNSS

```

1 # M_arr : (G1, G2, N, N) residual makers precomputados
2 # Sigma : (N, N)
3
4 sse_grid = np.einsum('ghjk,kj->gh', M_arr, Sigma) # (G1, G2)
5 sse_grid[~valid] = np.inf # marcar combinaciones invalidas
6 flat_best = np.argmin(sse_grid.ravel())
7 bi, bj = np.unravel_index(flat_best, (G1, G2))
8 tau1_t, tau2_t = tau1_grid[bi], tau2_grid[bj]
```

OLS SSE $\neq$ log-verosimilitud marginal

El Grid Search minimiza el SSE de OLS en dos etapas (Diebold-Li two-step), que puede seleccionar un τ distinto al optimo de la log-verosimilitud marginal del Filtro de Kalman. La diferencia puede ser de miles de puntos de log-lik. Sin embargo, el Grid Search es causalmente correcto y computacionalmente eficiente, lo que lo hace adecuado para el proposito de inicializar H_t . Los parametros del proceso de estado (F, Q, R, μ) se estiman correctamente por MLE en el paso siguiente.

4. Filtro de Kalman con H_t variable

4.1. Algoritmo completo

Algoritmo 4.1 (Kalman Filter con cargas variables — `_kf_core()`). Dado $\theta = (\mu, F, Q, R)$ y $\{H_t\}_{t=1}^T$, computar estados filtrados y log-verosimilitud.

Inicializacion: $\hat{\beta}_{0|0} = \mu$, $P_{0|0}$ estacionaria (ecuacion de Lyapunov).

Para $t = 1, \dots, T$:

Prediccion:

$$\hat{\beta}_{t|t-1} = \mu + F(\hat{\beta}_{t-1|t-1} - \mu) \quad (11)$$

$$P_{t|t-1} = F P_{t-1|t-1} F^\top + Q \quad (12)$$

Innovacion y varianza de innovacion:

$$v_t = y_t - H_t \hat{\beta}_{t|t-1} \quad (13)$$

$$S_t = H_t P_{t|t-1} H_t^\top + R \quad (14)$$

Contribucion a la log-verosimilitud (via descomposicion de Cholesky):

$$\ell_t = -\frac{N}{2} \log(2\pi) - \underbrace{\frac{1}{2} \sum_{n=1}^N 2 \log L_{t,nn}}_{\log |S_t|} - \frac{1}{2} \underbrace{\|L_t^{-1} v_t\|^2}_{v_t^\top S_t^{-1} v_t} \quad (15)$$

donde L_t es la factorizacion de Cholesky: $S_t = L_t L_t^\top$.

Actualizacion (forma Joseph):

$$K_t = P_{t|t-1} H_t^\top S_t^{-1} \quad (16)$$

$$\hat{\beta}_{t|t} = \hat{\beta}_{t|t-1} + K_t v_t \quad (17)$$

$$P_{t|t} = (I - K_t H_t) P_{t|t-1} (I - K_t H_t)^\top + K_t R K_t^\top \quad (18)$$

Log-verosimilitud total: $\log \mathcal{L} = \sum_{t=1}^T \ell_t$.

4.2. Por que la forma Joseph (ecuacion ??)

La forma estandar de actualizacion de covarianza es $(I - KH)P$, que equivale a la forma Joseph pero con menos operaciones. Sin embargo, la forma estandar pierde la simetria numerica de $P_{t|t}$ cuando la ganancia K_t es grande (filtro muy activo). La forma Joseph garantiza que:

$$P_{t|t} = P_{t|t}^\top \quad \text{y} \quad P_{t|t} \succ 0 \quad \text{en todo } t \quad (19)$$

Esto es esencial en series largas (TES Colombia con $T > 500$ observaciones diarias) donde los errores de redondeo de la forma estandar pueden acumularse y hacer $P_{t|t}$ indefinida.

4.3. Cholesky para log-det y cuadratica

El calculo del log-determinante y la forma cuadratica $\mathbf{v}_t^\top \mathbf{S}_t^{-1} \mathbf{v}_t$ nunca forma $\mathbf{S}_t^{-1}$ explicitamente. En cambio, se resuelve mediante la factorizacion de Cholesky:

$$\mathbf{S}_t = \mathbf{L}_t \mathbf{L}_t^\top \Rightarrow \log |\mathbf{S}_t| = 2 \sum_{n=1}^N \log(L_{t,nn}) \quad (20)$$

$$\mathbf{z}_t = \mathbf{L}_t^{-1} \mathbf{v}_t \Rightarrow \mathbf{v}_t^\top \mathbf{S}_t^{-1} \mathbf{v}_t = \|\mathbf{z}_t\|^2 \quad (21)$$

Esto es numericamente estable incluso cuando $\mathbf{S}_t$ esta mal condicionada. La ganancia de Kalman se calcula analogamente: $\mathbf{K}_t = \mathbf{S}_t^{-1} (\mathbf{H}_t \mathbf{P}_{t|t-1})^\top$ mediante dos triangular solves.

4.4. Inicializacion estacionaria

La covarianza inicial $\mathbf{P}_{0|0}$ se obtiene como la solucion de la ecuacion de Lyapunov discreta $\mathbf{P} = \mathbf{F} \mathbf{P} \mathbf{F}^\top + \mathbf{Q}$:

$$P_{0,ii} = \frac{q_i}{1 - f_i^2}, \quad P_{0,ij} = 0 \quad (i \neq j) \quad \text{para } \mathbf{F} \text{ diagonal} \quad (22)$$

Para $\mathbf{F}$ triangular o completa se usa `scipy.linalg.solve_discrete_lyapunov()`.

4.5. Implementacion con Numba JIT

La funcion `_kf_core()` esta compilada con Numba JIT para maxima velocidad. Acepta el array $(T \times N \times k)$ de cargas variables:

Celda 3 --- firma de `_kf_core()` y wrapper

```

1  # Firma del KF compilado (Numba JIT cuando esta disponible):
2  def _kf_core(yields,      # (T, N) rendimientos observados
3              H_arr,       # (T, N, k) cargas variables en t
4              F,           # (k, k) transicion
5              Q,           # (k, k) cov. innovaciones de estado
6              R_diag,      # (N,) varianzas de medicion
7              mu,          # (k,) media de largo plazo
8              beta0,       # (k,) estado inicial
9              P0):         # (k, k) covarianza inicial
10
11     # Retorna:
12     # beta_pred : (T, k)    beta_{t|t-1}
13     # beta_filt : (T, k)    beta_{t|t}
14     # P_pred    : (T, k, k) P_{t|t-1}
15     # P_filt    : (T, k, k) P_{t|t}
16     # innov     : (T, N)    innovaciones v_t
17     # loglik    : float     log-verosimilitud total
18     ...

```

5. Suavizador RTS (Rauch-Tung-Striebel)

5.1. Que aporta el suavizador

El Filtro de Kalman produce estados **filtrados** $\hat{\beta}_{t|t} = \mathbb{E}[\beta_t | \mathbf{Y}_{1:t}]$: la mejor estimación de β_t dado lo que se observa hasta t . Son causalmente correctos pero tienen mayor varianza porque no usan información futura.

El Suavizador RTS produce estados **suavizados** $\hat{\beta}_{t|T} = \mathbb{E}[\beta_t | \mathbf{Y}_{1:T}]$: la mejor estimación usando *toda* la muestra. Son preferibles para análisis histórico y diagnóstico in-sample.

5.2. Algoritmo backward

Algoritmo 5.1 (RTS Smoother — `rts_smoother()`). Usando los resultados del KF forward:

Inicialización: $\hat{\beta}_{T|T}$, $\mathbf{P}_{T|T}$ (del filtro).

Para $t = T - 1, T - 2, \dots, 1$:

$$\mathbf{G}_t = \mathbf{P}_{t|t} \mathbf{F}^\top \mathbf{P}_{t+1|t}^{-1} \quad (\text{ganancia suavizadora}) \quad (23)$$

$$\hat{\beta}_{t|T} = \hat{\beta}_{t|t} + \mathbf{G}_t (\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t}) \quad (24)$$

$$\mathbf{P}_{t|T} = \mathbf{P}_{t|t} + \mathbf{G}_t (\mathbf{P}_{t+1|T} - \mathbf{P}_{t+1|t}) \mathbf{G}_t^\top \quad (25)$$

El RTS es independiente de H_t

El suavizador RTS opera en el espacio de *estados* β_t , no en el espacio de observaciones. Por lo tanto, sus ecuaciones (??-??) solo dependen de $\mathbf{F}$ y $\mathbf{Q}$, y son válidas aunque las cargas H_t varíen en el tiempo. Esto es una propiedad elegante del modelo espacio-estado lineal gaussiano.

La ganancia suavizadora $\mathbf{G}_t = \mathbf{P}_{t|t} \mathbf{F}^\top \mathbf{P}_{t+1|t}^{-1}$ puede interpretarse como cuanta información futura (la diferencia $\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t}$) se propaga hacia atrás al revisar la estimación de β_t . Un $\mathbf{G}_t$ grande indica que el estado pasado debe revisarse significativamente dados los datos futuros.

6. Estimación MLE multi-start

6.1. Problema de optimizacion

Dados $\{H_t\}_{t=1}^T$ del Grid Search, los parametros del proceso de estado $\theta = (\mu, F, Q, R)$ se estiman maximizando la log-verosimilitud del Filtro de Kalman:

$$\hat{\theta} = \arg \max_{\theta} \log \mathcal{L}(\theta \mid \mathbf{Y}, \{H_t\}) = \arg \max_{\theta} \sum_{t=1}^T \ell_t(\theta) \quad (26)$$

El problema es no convexo: la superficie de log-verosimilitud puede tener multiples optimos locales, especialmente cuando F es triangular o completa. La solucion es el **multi-start**: ejecutar L-BFGS-B desde n_{starts} puntos aleatorios y retener el mejor.

6.2. Parametrizacion sin restricciones

Para garantizar que las restricciones de positividad y estabilidad siempre se cumplan durante la optimizacion, los parametros se transforman antes de pasarlos al optimizador:

Table 4: Reparametrizacion para la optimizacion sin restricciones

Parametro	Restriccion	Transformacion	Inversa
f_{ii} (diagonal F)	$ f_{ii} < 0.995$	$x_i = \text{arctanh}(f_{ii}/0.995)$	$f_{ii} = 0.995 \tanh(x_i)$
f_{ij} ($i > j$, triangular)	libre	$x_{ij} = f_{ij}$	$f_{ij} = x_{ij}$
q_i (diagonal Q)	$q_i > 0$	$x_i = \log(q_i)$	$q_i = e^{x_i}$
r_n (diagonal R)	$r_n > 0$	$x_n = \log(r_n)$	$r_n = e^{x_n}$
μ_i	libre	$x_i = \mu_i$	$\mu_i = x_i$

El limite $|f_{ii}| < 0.995$ (no exactamente 1) garantiza que los eigenvalores de $\mathbf{F}$ esten dentro del circulo unitario, asegurando estacionariedad del proceso de estado. Usar $\tanh$ en lugar de clip produce gradientes continuos que el optimizador puede usar.

6.3. Numero de parametros por configuracion

Table 5: Parametros libres en la MLE segun model y F_type

model	k	F_type	Params en F	Params μ, Q, R	Total
DNS	3	diagonal	3	$3 + 3 + N$	$9 + N$
DNS	3	triangular	6	$3 + 3 + N$	$12 + N$
DNS	3	completa	9	$3 + 3 + N$	$15 + N$
DNSS	4	diagonal	4	$4 + 4 + N$	$12 + N$
DNSS	4	triangular	10	$4 + 4 + N$	$18 + N$
DNSS	4	completa	16	$4 + 4 + N$	$24 + N$

Criterios de informacion usados para comparar modelos:

$$\text{AIC} = -2 \log \mathcal{L} + 2p \quad (27)$$

$$\text{BIC} = -2 \log \mathcal{L} + p \log T \quad (28)$$

donde p es el numero de parametros libres.

6.4. Extensiones para plazos largos

6.4.1. Opcion A — R estructurada por grupos

Reduce N varianzas a 3 parametros (corto/medio/largo):

$$r_n = r_{g(n)}, \quad g(n) = \begin{cases} \text{corto} & m_n \leq \text{th}_1 \\ \text{medio} & \text{th}_1 < m_n \leq \text{th}_2 \\ \text{largo} & m_n > \text{th}_2 \end{cases} \quad (29)$$

En el MLE se optimizan solo 3 parametros de R en lugar de N , reduciendo el riesgo de sobreajuste y reflejando la estructura economica del mercado (los spreads de liquidez varian por tramo).

6.4.2. Opcion C — Pesos en la log-verosimilitud

Escala inversamente las varianzas de medicion antes del KF: $r_n^{\text{eff}} = r_n/w_n$. Un peso $w_n > 1$ hace que el filtro confie mas en el plazo n , lo que fuerza al MLE a ajustarlo mejor:

$$\log \mathcal{L}^{(w)} = \sum_{t=1}^T \sum_{n=1}^N -\frac{1}{2} \left[\log(2\pi r_n/w_n) + \frac{v_{t,n}^2}{r_n/w_n} \right] \quad (30)$$

Esta es una cuasi-maxima verosimilitud ponderada: el investigador asigna mayor importancia a ajustar bien ciertos plazos.

7. Analisis post-ajuste

El notebook genera automaticamente seis bloques de diagnostico, todos accesibles tambien de forma individual.

7.1. Factores latentes — `plot_factores()`

Grafica los estados **filtrados** $\hat{\beta}_{t|t}$ y **suavizados** $\hat{\beta}_{t|T}$ para cada factor con bandas de confianza al 95% basadas en la diagonal de $\mathbf{P}_{t|T}$:

$$\hat{\beta}_{i,t|T} \pm 1.96 \sqrt{P_{t|T,ii}} \quad (31)$$

El diferencial entre filtrado y suavizado es diagnosticamente util: una diferencia grande indica que el modelo revisa sustancialmente su estimacion del estado pasado al recibir datos futuros, lo que puede senalar mal especificacion en la dinamica de transicion.

Se reportan ademas:

- Estadisticos descriptivos: media, std, percentiles.
- Autocorrelaciones a lag 1 y lag 5 como medida de persistencia.

7.2. Evolucion de τ_t — `plot_tau_evolution()`

Muestra la serie temporal de τ_t con la ubicacion del hump $m_t^* = 1.793 \tau_t$ en el eje secundario. Incluye banda de percentiles P10–P90 y distribucion marginal con KDE.

Una alta variabilidad en τ_t indica que la forma de la curva cambia significativamente en el tiempo. Para TES Colombia esto es esperado dado los distintos regimenes monetarios del Banco de la Republica.

7.3. Curva ajustada — `plot_curvas_ajustadas()`

Superpone la curva observada (puntos) con la curva ajustada (linea continua) en n fechas seleccionadas uniformemente. La curva ajustada se evalua en una malla fina de 200 puntos usando el estado suavizado: $\hat{y}_t(m) = \mathbf{H}(m; \tau_t)^\top \hat{\beta}_{t|T}$.

7.4. Residuos de medicion — `analisis_residuos()`

Cuatro niveles de diagnostico:

1. **RMSE y MAE por plazo:** identifica donde falla el modelo. Tipicamente los plazos cortos (poca liquidez) y largos (prima de plazo) tienen mayor error que los intermedios.
2. **Sesgo medio por plazo:** sesgo positivo = subestimacion sistematica. Un sesgo grande en el plazo largo indica que la estructura factorial no captura la prima de plazo.
3. **Correlacion cruzada de residuos:** si los residuos de distintos plazos estan correlacionados, el modelo omite un factor comun. Alta correlacion entre residuos 20Y y 30Y es tipica cuando hay una prima de plazo no modelada.
4. **Tests estadisticos:** Jarque-Bera (normalidad) y Ljung-Box lag 10 (autocorrelacion serial) por plazo. LB $p < 0.05$ indica patrones seriales residuales que sugieren mal especificacion.

7.5. Diagnosticos del filtro — `diagnosticos_filtro()`

Condicion de especificacion correcta del KF

Un Filtro de Kalman correctamente especificado produce innovaciones **blancas y normalizadas**:

$$\tilde{v}_{t,n} = \frac{v_{t,n}}{\hat{\sigma}_n} \sim \mathcal{N}(0, 1) \quad \text{i.i.d.} \quad (32)$$

Si hay autocorrelacion en las innovaciones, el modelo esta mal especificado (por ejemplo, la dinamica de transicion o las cargas son insuficientemente flexibles).

Se reportan:

- Series temporales de $\tilde{v}_{t,n}$ con marcado de outliers ($|\tilde{v}| > 2.58\sigma$).
- Histograma con curva $\mathcal{N}(0, 1)$ teorica.
- Log-verosimilitud acumulada: una ruptura en su pendiente puede indicar un cambio de regimen.
- Test de Ljung-Box lag 10 sobre las innovaciones normalizadas.

7.6. Parametros MLE — `resumen_parametros()`

- μ : media de largo plazo de los factores. Bajo especificacion correcta, β_t revierte a μ en el largo plazo.
- **F**: heatmap con eigenvalores graficados en el circulo unitario. Un eigenvalor fuera del circulo indica inestabilidad.
- **Semividas implicitas**: $h_i = -\log(2)/\log(|f_{ii}|)$ periodos. Indica cuanto tarda cada factor en reducir su desviacion respecto a μ_i a la mitad.
- $\sigma_{\eta,i} = \sqrt{q_i}$ y $\sigma_{\varepsilon,n} = \sqrt{r_n}$ en puntos basicos.

7.7. Perfil SSE de τ — `plot_perfil_sse_tau()`

Heatmap del perfil SSE normalizado en el tiempo para DNS: los candidatos $\tau \in \mathcal{G}$ en el eje vertical, el tiempo en el eje horizontal, y la intensidad representa el SSE (amarillo = bajo SSE = candidato preferido). Superpone la trayectoria $\hat{\tau}_t$ seleccionada en cada periodo. Diagnostica la robustez de la seleccion de τ : un perfil SSE muy plano indica que muchos valores de τ tienen ajuste similar, lo que sugiere poca identificacion de la curvatura en ese periodo.

8. Documentacion de funciones

8.1. Cargas

Celda 2 --- `ns_loadings()`, `nss_loadings()`, `get_H()`

```

1 def ns_loadings(maturities, tau1):
2     '''Cargas Nelson-Siegel. Shape (N, 3). [nivel, pendiente, curvatura]'''
3     m = np.asarray(maturities, dtype=float)
4     mt = m / tau1; e = np.exp(-mt); c = (1.0 - e) / mt
5     return np.column_stack([np.ones_like(m), c, c - e])
6
7 def nss_loadings(maturities, tau1, tau2):
8     '''Cargas Nelson-Siegel-Svensson. Shape (N, 4).
9     Requiere |tau1 - tau2| >= 0.15 para evitar colinealidad.'''

```

```

10     ...
11
12 def get_H(maturities, tau1, tau2, model):
13     '''Wrapper: model="DNS" -> ns_loadings | "DNSS" -> nss_loadings.'''
14     ...

```

8.2. Grid Search

Celda 5 --- Grid Search de tau_t

```

1  # Residual maker (precomputado una vez por candidato):
2  M = _residual_maker(H)                #  $M = I - H(H'H)^{-1}H'$ , shape (N,
    N)
3
4  # Sumas acumuladas de y_s y_s' (precomputadas una vez):
5  YYt_cum = _build_YYt_cumsum(yields) # shape (T+1, N, N)
6
7  # DNS: Grid Search 1D
8  tau1_t, sse_t, sse_grid = rolling_tau_dns(
9      yields, maturities,
10     tau1_grid = np.linspace(0.5, 5.0, 35),
11     window_size = 52)                # None -> ventana expandiente
12
13 # DNSS: Grid Search 2D
14 tau1_t, tau2_t, sse_t = rolling_tau_dnss(
15     yields, maturities,
16     tau1_grid = np.linspace(0.5, 5.0, 22),
17     tau2_grid = np.linspace(0.15, 3.0, 18),
18     window_size = 52,
19     sep_min = 0.15)                # separacion minima |tau1 - tau2|
20
21 # Construir array de cargas (T, N, k) a partir de tau_t:
22 H_arr = build_H_array(maturities, tau1_t, tau2_t, model='DNSS')

```

8.3. Kalman Filter y RTS

Celdas 3-4 --- KF y RTS

```

1  # Kalman Filter con H_t variable:
2  beta_pred, beta_filt, P_pred, P_filt, innov, loglik = _kf_core(
3      yields,      # (T, N)
4      H_arr,      # (T, N, k) cargas variables
5      F,          # (k, k)
6      Q,          # (k, k) --- diagonal
7      R_diag,     # (N,)
8      mu,         # (k,)
9      beta0,      # (k,)
10     P0)         # (k, k)

```



```

11
12 # Condicion inicial estacionaria:
13 P0 = compute_P0_stationary(F, Q, F_type='diagonal')
14
15 # RTS Smoother (backward):
16 beta_smooth, P_smooth, G_gain = rts_smoother(
17     beta_filt, P_filt,
18     beta_pred, P_pred, F)

```

8.4. MLE

Celda 6 --- empaquetado y MLE multi-start

```

1 # Empaquetar parametros en vector para el optimizador:
2 params = pack_params(mu, F, Q_diag, R_diag, k, F_type)
3
4 # Desempaquetar con transformaciones inversas:
5 mu, F, Q, Q_diag, R_diag = unpack_params(params, k, N, F_type)
6
7 # Funcion objetivo (negativa para minimizacion):
8 neg_ll = neg_loglik(params, yields, H_arr, k, N, F_type)
9
10 # MLE completo multi-start:
11 mu, F, Q, R_diag, loglik, best_params = mle_estimate(
12     yields, H_arr, k, N,
13     F_type = 'diagonal',
14     n_starts = 6,           # intentos con puntos de inicio aleatorios
15     verbose = True)

```

8.5. Pipeline completo — fit_model()

Celda 7 --- fit_model() interno

```

1 res = fit_model(
2     yields, maturities, dates,
3     model      = 'DNS',
4     F_type     = 'diagonal',
5     tau1_grid  = np.linspace(0.5, 5.0, 35),
6     tau2_grid  = None,           # solo para DNSS
7     window_size = 52,          # None -> expandiente
8     n_starts   = 6,
9     verbose    = True,
10 )
11 # Retorna dict con todos los resultados (ver Seccion 8.3)

```

8.6. Analisis post-ajuste

Celdas 11-17 --- graficos y diagnosticos

```

1 # Factores (filtrado vs suavizado con IC 95%):
2 plot_factores(res, titulo_extra='(DNS)')
3
4 # Evolucion temporal de tau:
5 plot_tau_evolution(res)
6
7 # Curva ajustada en n fechas seleccionadas:
8 plot_curvas_ajustadas(res, n_fechas=6)
9
10 # Heatmap de residuos en el tiempo:
11 plot_heatmap_residuos(res)
12
13 # Analisis completo de residuos (RMSE, sesgo, correlaciones, tests):
14 analisis_residuos(res)
15
16 # Diagnosticos del filtro (innovaciones, ACF, log-lik acumulada):
17 diagnosticos_filtro(res)
18
19 # Resumen de parametros MLE (F, eigenvalores, semividas, sigma_eps):
20 resumen_parametros(res)
21
22 # Perfil SSE de tau en el tiempo (solo DNS):
23 plot_perfil_sse_tau(res)

```

9. Funcion principal fit_dns_kalman()

9.1. Firma completa

Celda 18 --- firma de fit_dns_kalman()

```

1 def fit_dns_kalman(
2     dates,                # DatetimeIndex o array de fechas
3     yields,               # (T, N) DataFrame o ndarray
4     maturities,           # (N,) plazos en anos
5     # Modelo
6     model                  = 'DNS',      # 'DNS' (k=3) o 'DNSS' (k=4)
7     F_type                 = 'diagonal', # 'diagonal'/'triangular'/'completa'
8     # Grid Search de tau_t
9     tau1_grid              = None,       # candidatos tau1 (None -> auto 35
        pts)

```

```

10     tau2_grid      = None,          # candidatos tau2 (None -> auto 20
    pts)
11     window_size    = None,          # int -> rodante / None ->
    expandiente
12     sep_min_dnss    = 0.15,         # separacion minima |tau1-tau2| (
    DNSS)
13     # MLE
14     n_starts        = 5,            # intentos multi-start
15     obs_weights     = None,         # (N,) Opcion C: pesos por plazo
16     r_mode          = 'libre',      # 'libre' / 'grupos' (Opcion A)
17     group_thresholds = (3.0, 10.0), # fronteras de grupos en anos
18     # Control
19     plot            = True,          # generar todos los graficos
20     verbose         = True,          # imprimir progreso
21 )

```

9.2. Descripcion de parametros

Parametro	Default	Descripcion
model	'DNS'	'DNS': Nelson-Siegel, $k = 3$ factores (β_0 =nivel, β_1 =pendiente, β_2 =curvatura). 'DNSS': Svensson, $k = 4$ factores (+ β_3 =segunda curvatura).
F_type	'diagonal'	Estructura de la matriz de transicion $\mathbf{F}$. 'diagonal': AR(1) independiente por factor (recomendado por literatura). 'triangular': permite spillovers jerarquicos. 'completa': VAR(1) irrestricto. Verificar estabilidad post-estimacion.
tau1_grid	None	Candidatos para τ_1 en el Grid Search. None genera automaticamente <code>linspace(0.5, 5.0, 35)</code> . Ampliar el grid si $\hat{\tau}_t$ tiende a caer en los extremos del rango.
tau2_grid	None	Candidatos para τ_2 (solo DNSS). None genera <code>linspace(0.15, 3.0, 20)</code> . Solo relevante cuando <code>model='DNSS'</code> .
window_size	None	Tamano de ventana para el Grid Search. None: ventana expandiente (usa todo $[1, t]$; causal; recomendada para pronostico). Entero W : ventana rodante de los ultimos W periodos (mas reactiva).

Parametro	Default	Descripcion
n_starts	5	Intentos de la optimizacion MLE con distintos puntos de inicio. Mas intentos $\rightarrow$ mayor probabilidad de optimo global, mayor tiempo de computo. Recomendado 6–8 para produccion.
obs_weights	None	Opcion C. Array $(N,)$ de pesos por plazo para la log-verosimilitud. None = todos iguales. Plazos con $w_n > 1$ reciben menor varianza de medicion efectiva $\rightarrow$ mejor ajuste. Ejemplo: <code>np.where(maturities >= 10, 2.5, 1.0)</code> .
r_mode	'libre'	Opcion A. 'libre': estima N varianzas de medicion independientes. 'grupos': estima solo 3 varianzas por grupos de plazos, definidos por <code>group_thresholds</code> .
group_thresholds	(3.0, 10.0)	Fronteras de grupos en anos para <code>r_mode='grupos'</code> . Default: corto $\leq 3Y$, medio $\leq 10Y$, largo $> 10Y$.
plot	True	Si True, genera todos los graficos de diagnostico: factores, τ_t , curvas ajustadas, heatmap, residuos, diagnosticos KF, parametros MLE, perfil SSE.

9.3. Estructura del retorno

La funcion retorna un diccionario con todos los resultados del pipeline:

Claves del dict de retorno

```

1 res = fit_dns_kalman(...)
2
3 # Modelo y configuracion
4 res['model']          # str      'DNS' o 'DNSS'
5 res['F_type']         # str      tipo de F usado
6 res['window_size']    # int/None tamaño de ventana
7
8 # Secuencias adaptativas de tau
9 res['tau1_t']         # (T,)    tau1 optimo por periodo
10 res['tau2_t']         # (T,)    tau2 optimo por periodo (DNSS; NaN para
    DNS)
11 res['sse_grid']       # (T, G1) perfil SSE completo (DNS; None para
    DNSS)
12 res['H_arr']          # (T, N, k) cargas variables en t
13
14 # Parametros MLE

```

```

15 res['mu']           # (k,)      media de largo plazo estimada
16 res['F']           # (k, k)    matriz de transicion estimada
17 res['Q']           # (k, k)    cov. innovaciones de estado
18 res['R_diag']      # (N,)      varianzas de medicion
19
20 # Estados del Filtro de Kalman
21 res['beta_filtered'] # (T, k)    estados filtrados  $\beta_{t/t}$ 
22 res['P_filtered']   # (T, k, k) covarianzas filtradas  $P_{t/t}$ 
23 res['beta_predicted'] # (T, k)    estados predichos  $\beta_{t/t-1}$ 
24 res['P_predicted']   # (T, k, k) covarianzas predichas  $P_{t/t-1}$ 
25 res['innovations']  # (T, N)    innovaciones  $v_t$ 
26
27 # Estados del RTS Smoother
28 res['beta_smoothed'] # (T, k)    estados suavizados  $\beta_{t/T}$ 
29 res['P_smoothed']   # (T, k, k) covarianzas suavizadas  $P_{t/T}$ 
30 res['G_gain']       # (T, k, k) ganancias suavizadoras  $G_t$ 
31
32 # Ajuste y residuos
33 res['fitted']        # (T, N)    rendimientos ajustados  $H_t @ \beta_{t/T}$ 
34 res['residuals']     # (T, N)    residuos  $y_t - \text{fitted}_t$ 
35
36 # Metricas de ajuste
37 res['loglik']         # float    log-verosimilitud total
38 res['aic']           # float    AIC =  $-2 \cdot \text{loglik} + 2 \cdot p$ 
39 res['bic']           # float    BIC =  $-2 \cdot \text{loglik} + p \cdot \log(T)$ 
40 res['rmse_total']    # float    RMSE global (mismas unidades que yields
    )
41 res['mae_total']     # float    MAE global
42 res['rmse_mat']      # (N,)     RMSE por plazo
43 res['mae_mat']       # (N,)     MAE por plazo
44
45 # Datos originales
46 res['maturities']    # (N,)     plazos en anos
47 res['dates']         # DatetimeIndex
48 res['yields']        # (T, N)    rendimientos originales

```

10. Ejemplos de uso

10.1. Ejemplo 1 — DNS basico

Uso minimo --- DNS con F diagonal y ventana expandiente

```

1 import numpy as np
2 import pandas as pd
3
4 df = pd.read_csv('rendimientos.csv', parse_dates=['Date'])

```

```

5 df = df.set_index('Date')
6 yield_cols = ['Y1', 'Y2', 'Y3', 'Y5', 'Y10', 'Y20']
7 mats      = np.array([1, 2, 3, 5, 10, 20], dtype=float)
8
9 res = fit_dns_kalman(
10     dates      = df.index,
11     yields     = df[yield_cols].values,
12     maturities = mats,
13     model      = 'DNS',
14     F_type     = 'diagonal',
15     window_size= None,      # ventana expandiente (causal)
16     n_starts   = 6,
17 )
18
19 # Resultados basicos:
20 print(f"RMSE = {res['rmse_total']*100:.3f} pb")
21 print(f"AIC   = {res['aic']:.1f}")
22 print(f"tau1: min={res['tau1_t'].min():.3f}  "
23       f"med={np.median(res['tau1_t']):.3f}  "
24       f"max={res['tau1_t'].max():.3f}")
25
26 # Extraer factores suavizados:
27 beta_t      = res['beta_smoothed']  # (T, 3)
28 nivel       = beta_t[:, 0]          # nivel (tasa de largo plazo)
29 pendiente   = beta_t[:, 1]          # spread largo-corto
30 curvatura   = beta_t[:, 2]          # factor de curvatura

```

10.2. Ejemplo 2 — DNSS con ventana rodante

DNSS con tau adaptativo por ventana

```

1 res = fit_dns_kalman(
2     dates      = df.index,
3     yields     = df[yield_cols].values,
4     maturities = mats,
5     model      = 'DNSS',
6     F_type     = 'diagonal',
7     tau1_grid  = np.linspace(0.5, 5.0, 25),
8     tau2_grid  = np.linspace(0.15, 3.0, 20),
9     window_size= 52,      # ventana de 1 ano (datos semanales)
10    n_starts   = 6,
11 )
12
13 # Acceder a factores con incertidumbre:
14 bs  = res['beta_smoothed']  # (T, 4) media posterior
15 Ps  = res['P_smoothed']    # (T, 4, 4) covarianza
16 sd_b = np.sqrt(np.maximum(Ps[:, :, :].diagonal(axis1=1, axis2=2), 0))
17 ic95_lo = bs - 1.96 * sd_b
18 ic95_hi = bs + 1.96 * sd_b

```

10.3. Ejemplo 3 — Mejoras para plazos largos (Opciones A y C)

Opciones A y C combinadas

```

1 mats    = np.array([1, 2, 3, 5, 10, 20], dtype=float)
2 pesos    = np.where(mats >= 10, 2.5, 1.0)  # peso extra a plazos largos
3
4 res = fit_dns_kalman(
5     dates          = df.index,
6     yields         = df[yield_cols].values,
7     maturities     = mats,
8     model          = 'DNSS',
9     F_type         = 'diagonal',
10    window_size    = 52,
11    n_starts       = 6,
12    obs_weights     = pesos,                # Opcion C: pesos
13    r_mode          = 'grupos',            # Opcion A: R por grupos
14    group_thresholds = (3.0, 10.0),
15 )
16
17 # Comparar RMSE por plazo vs modelo base:
18 for i, m in enumerate(mats):
19     print(f"    {m:.0f}Y: RMSE = {res['rmse_mat'][i]*100:.3f} pb")

```

10.4. Ejemplo 4 — Comparar las tres estructuras de F

Experimento: impacto del tipo de F

```

1 # Correr los tres tipos de F sin graficos intermedios:
2 resultados = {}
3 for f_type in ['diagonal', 'triangular', 'completa']:
4     r = fit_dns_kalman(
5         dates=df.index, yields=df[yield_cols].values,
6         maturities=mats, model='DNS',
7         F_type=f_type, window_size=52, n_starts=4,
8         plot=False, verbose=False)
9     resultados[f_type] = r
10
11 # Tabla comparativa:
12 print(f"    {'F_type':<12} {'RMSE(pb)':>9} {'Log-lik':>12} "
13       f"{'AIC':>10} {'BIC':>10}")
14 for ft, r in resultados.items():
15     print(f"    {ft:<12} {r['rmse_total']*100:>9.4f} "
16           f"{r['loglik']:>12,.1f} {r['aic']:>10,.1f} {r['bic']:>10,.1f} "
17           ")
18
19 # Graficos completos del mejor modelo por AIC:
20 best = min(resultados, key=lambda k: resultados[k]['aic'])

```

```

20 res_best = resultados[best]
21 plot_factores(res_best, titulo_extra=f"[F_type='{best}']")
22 plot_tau_evolution(res_best)
23 analisis_residuos(res_best)
24 resumen_parametros(res_best)

```

10.5. Ejemplo 5 — Uso programatico para comparativa con Bayes

Comparar KF con el notebook bayesiano

```

1  # Ajuste KF
2  res_kf = fit_dns_kalman(
3      dates=df.index, yields=df[yield_cols].values,
4      maturities=mats, model='DNS',
5      plot=False, verbose=False)
6
7  # Ajuste Bayesiano (del otro notebook):
8  # res_bayes = fit_dns_bayes(...)
9
10 # Comparar RMSE:
11 for i, m in enumerate(mats):
12     r_kf = res_kf['rmse_mat'][i] * 100
13     r_bayes = res_bayes['fit']['rmse_mat'][i] * 100
14     print(f" {m:.0f}Y KF={r_kf:.3f} Bayes={r_bayes:.3f} "
15           f"Delta={r_bayes-r_kf:+.3f}")
16
17 # Ancho del IC de beta_0:
18 # KF: basado en P_smooth de RTS (gaussiano, condicional en theta_hat)
19 # Bayes: percentiles de muestras MCMC (integra incertidumbre de theta y
    tau)
20 sd_kf = np.sqrt(res_kf['P_smoothed'][:, 0, 0])
21 ancho_kf = (2 * 1.96 * sd_kf).mean()
22
23 b_mean, q025, q975, _ = extract_posterior_betas(res_bayes['chains'])
24 ancho_bayes = (q975[:, 0] - q025[:, 0]).mean()
25
26 print(f"\nIC medio beta_0 --- KF: {ancho_kf:.4f} Bayes: {ancho_bayes
    :.4f}")
27 print("El IC bayesiano es mas amplio: integra incertidumbre de theta y
    tau.")

```

11. Referencia completa de funciones

Table 7: Todas las funciones del notebook en orden de dependencia

Funcion	Retorno / Forma	Descripcion
ns_loadings(m, tau1)	$(N, 3)$	Cargas Nelson-Siegel
nss_loadings(m, tau1, tau2)	$(N, 4)$	Cargas Nelson-Siegel-Svensson
get_H(m, tau1, tau2, model)	(N, k)	Wrapper unificado NS/NSS
_residual_maker(H)	(N, N)	$M = I - H(H^\top H)^{-1}H^\top$; None si $\text{cond} > 10^{12}$
_build_YYt_cumsum(yields)	$(T + 1, N, N)$	Sumas acumuladas $\Sigma_t = \sum_{s \leq t} \mathbf{y}_s \mathbf{y}_s^\top$
rolling_tau_dns(y, m, grid, W)	$(T,), (T,), (T, G)$	Grid Search 1D de $\tau_{1,t}$ para DNS
rolling_tau_dnss(y, m, g1, g2, W)	$(T,), (T,), (T,)$	Grid Search 2D de $(\tau_{1,t}, \tau_{2,t})$ para DNSS
build_H_array(m, tau1_t, tau2_t, model)	(T, N, k)	Construye array de cargas desde secuencias τ_t
compute_P0_stationary(F, Q, F_type)	(k, k)	Condicion inicial P_0 via Lyapunov discreta
_kf_core(y, H_arr, F, Q, R, mu, b0, P0)	multiples	Kalman Filter con H_t variable (Numba JIT)
rts_smoother(bf, Pf, bp, Pp, F)	$(T, k), (T, k, k), (T, k, k)$	Suavizador RTS (backward)
pack_params(mu, F, Q, R, k, F_type)	$(p,)$	Empaqueta parametros con reparametrizacion
unpack_params(params, k, N, F_type)	multiples	Invierte pack_params
neg_loglik(params, y, H, k, N, F_type)	float	Funcion objetivo para L-BFGS-B
_random_init_params(k, N, F_type, seed)	$(p,)$	Punto de inicio aleatorio para multi-start
mle_estimate(y, H, k, N, F_type, n_starts)	multiples	MLE multi-start L-BFGS-B
fit_model(y, m, dates, model, F_type, ...)	dict	Pipeline completo (Grid Search + MLE + KF + RTS)
plot_factores(res)	—	Factores filtrados/suavizados con IC 95%
plot_tau_evolution(res)	—	Serie temporal de τ_t y hump location
plot_curvas_ajustadas(res, n_fechas)	—	Curva observada vs ajustada en n fechas
plot_heatmap_residuos(res)	—	Heatmap de residuos en el tiempo por plazo
analisis_residuos(res)	—	RMSE/MAE/sesgo, correlaciones, QQ, ACF, tests
diagnosticos_filtro(res)	—	Innovaciones normalizadas, blancura
resumen_parametros(res)	—	F , eigenvalores, semividas, σ_ε
plot_perfil_sse_tau(res)	—	Heatmap $\text{SSE}(\tau, t)$ en el tiempo (DNS)
fit_dns_kalman(dates, yields, mats, ...)	dict	Funcion principal publica

12. Guia de decisiones

Table 8: Que configuracion usar segun el caso

Situacion	Configuracion recomendada	Justificacion
Prototipado rapido	n_starts=3, window_size=None	Verificar que el modelo corre
Produccion / publicacion	n_starts=6, window_size=52	Balance entre robustez y reactividad
Curva con una joroba	model='DNS'	Parsimonia, AIC tipicamente mejor
Curva con prima de plazo en largo	model='DNSS'	Segundo factor de curvatura necesario
Estructura de F	F_type='diagonal' inicialmente	Consenso empirico; probar 'completa' si AIC mejora > 5
Residuos grandes en plazos largos	obs_weights + r_mode='grupos'	Opciones A y C combinadas
τ cae en el extremo del grid	Ampliar tau1_grid	El optimo puede estar fuera del rango
Alta varianza de τ_t	Aumentar window_size	Mas datos $\rightarrow \tau_t$ mas estable
Pronostico fuera de muestra	window_size=None	Ventana expandiente es causal
Comparar con enfoque bayesiano	Mismo model y F_type	KF es referencia rapida; Bayes da incertidumbre completa
TES Colombia (2019–2026)	model='DNSS', window_size=52	Multiples regimenes BanRep; segundo factor util

Referencias

- Nelson, C.R. & Siegel, A.F. (1987). Parsimonious Modeling of Yield Curves. *Journal of Business*, 60(4), 473–489.
- Svensson, L.E.O. (1994). Estimating and Interpreting Forward Interest Rates: Sweden 1992–1994. *NBER Working Paper* No. 4871.
- Diebold, F.X. & Li, C. (2006). Forecasting the Term Structure of Government

Bond Yields. *Journal of Econometrics*, 130(2), 337–364.

- **Diebold, F.X. & Rudebusch, G.D.** (2013). *Yield Curve Modeling and Forecasting: The Dynamic Nelson-Siegel Approach*. Princeton University Press.
- **Caldeira, J.F., Laurini, M.P. & Portugal, M.S.** (2010). Bayesian Inference Applied to Dynamic Nelson-Siegel Model. *Brazilian Review of Econometrics*, 30(1).
- **Cakmaklı, C.** (2013). Modelling the Term Structure with a Bayesian Dynamic Nelson-Siegel Model. Erasmus University Rotterdam Working Paper.
- **Hamilton, J.D.** (1994). *Time Series Analysis*. Princeton University Press. Capítulo 13: State-Space Models and the Kalman Filter.
- **Rauch, H.E., Tung, F. & Striebel, C.T.** (1965). Maximum Likelihood Estimates of Linear Dynamic Systems. *AIAA Journal*, 3(8), 1445–1450.
- **BIS** (2005). Zero-Coupon Yield Curves: Technical Documentation. *BIS Papers* No. 25, Bank for International Settlements.